

Chapter 9

Machine Learning for Protein Engineering



Kadina E. Johnston, Clara Fannjiang, Bruce J. Wittmann, Brian L. Hie, Kevin K. Yang, and Zachary Wu

Abstract Directed evolution of proteins has been the most effective method for protein engineering. However, a new paradigm is emerging, fusing the library generation and screening approaches of traditional directed evolution with computation through the training of machine learning models on protein sequence fitness data. This chapter highlights successful applications of machine learning to protein engineering and directed evolution, organized by the improvements that have been made with respect to each step of the directed evolution cycle. Additionally, we provide an outlook for the future based on the current direction of the field, namely in the development of calibrated models and in incorporating other modalities, such as protein structure.

Keywords Protein engineering · Directed evolution · Enzyme engineering · Sequence models · Machine learning · Deep learning

All authors contributed equally.

K. E. Johnston · B. J. Wittmann
California Institute of Technology, Pasadena, USA

C. Fannjiang
University of California, Berkeley, USA

B. J. Wittmann
Currently at Microsoft, Redmond, USA

B. L. Hie
Stanford University, Stanford, USA

K. K. Yang
Microsoft Research New England, Cambridge, USA

Z. Wu (✉)
Currently at DeepMind, London, UK
e-mail: molzacharywu@gmail.com

Proteins are the molecular drivers for many of the processes necessary for life, performing functions such as binding small molecules, stabilizing other proteins, or catalyzing vital reactions. They are synthesized as linear sequences of individual building blocks called amino acids, of which there are twenty canonical types; these sequences fold into complex and dynamic three-dimensional structures to accomplish their functions. Existing proteins perform diverse functions, but nature only selects for a small fraction of them, leaving many functions unexplored, including many in which humans are interested.

Thus, protein engineers pursue these functions, most often with a powerful functional optimization strategy known as directed evolution. Directed evolution adapts nature's evolutionary process to enable the rapid improvement of proteins for human-desired functions [1, 2]. Specifically, directed evolution iterates between modifying proteins at the sequence level and identifying protein sequences with improved level of function. This process has yielded many impactful engineered proteins, ranging from antibodies for combating disease to enzymes capable of catalyzing non-natural reactions, and its success has been recognized by the 2018 Nobel Prize in Chemistry.

Fundamentally, directed evolution is an optimization process for protein sequences, and historically, it has been relatively straightforward but effective. More recently, advances in machine learning have been successful in improving directed evolution's ability to identify better proteins. In this chapter, we review the core concepts that have enabled successful integration of machine learning into directed evolution and protein engineering at-large by interpreting the process through the directed evolution cycle.

We begin with background on directed evolution, outlining the steps involved and casting them in the context of navigating protein fitness landscapes. We then highlight recent developments in machine learning's contribution to each of these steps as well as the experimental constraints, and finally we end with an outlook for the field. We focus on methods employing sequence-based machine learning for the directed evolution of proteins, but there are many other aspects of protein modeling that can be explored in biology [3] and chemistry that we leave out [4]. For example, proteins do not function in a vacuum, and other molecules, such as the nucleotides that encode our genomes [5, 6] and small molecules [7], can have profound effects on proteins' biological functions. Another view of understanding proteins is through their three-dimensional structures [8]. Additionally, protein dynamics [9] and structure prediction [10] are two rapidly growing fields, but the connection to protein function remains opaque [11]. Nevertheless, the goal of protein structure-based design [12] is to replace much of the work required for protein engineering, instead generating optimized proteins in a single step. While these approaches hold much promise, particularly in light of the paradigm-shifting success of AlphaFold [13], a protein structure prediction network, much of the established work in protein engineering is rooted in protein sequence.

9.1 Background

Directed evolution is an adaption of natural (and artificial) selection to the modern laboratory. First, consider a simplified view of how a new protein or function may arise naturally. In nature, evolution occurs within populations and is largely driven by competition to survive, grow, and reproduce. Therefore, an attribute which confers an advantage for any of these processes is selected for, enabling an organism to out-compete the organisms around it. The advantage could come from a mutation in a protein sequence that imparts a low level of activity for a new, beneficial function, making the organism and its offspring more successful. Thus, this protein sequence becomes more common as well. Through many iterations of this process, that protein sequence and its function could be further optimized. This process is similar to many engineering strategies, but contains notable distinctions for proteins, discussed in the following Sects. 9.1.1 and 9.1.1.1.

9.1.1 Protein Sequence Space

The number of possible protein sequences is immense, presenting a significant challenge to any protein engineering strategy. A typical protein is several hundred amino acids long, and at each position there are twenty canonical amino acids possible. This results in 20^{300} (10^{390}) possible sequences, which is around 10^{300} times more than the total number of atoms in the universe. There will never be enough resources to synthesize all of these sequences let alone screen them.

Interestingly, this does not mean that finding functional proteins or improving protein fitness is a hopeless endeavor. Maynard Smith made the critical observation that for natural selection to be possible, functional protein sequences must neighbor other functional sequences in protein space, implying that functional proteins exist clustered together within a vast sea of non-functional ones [14]. Therefore, rather than throwing metaphorical darts at this astronomical space and hoping for a hit, protein engineers can begin with a small level of activity for their function of interest and leverage methods for local exploration to improve it.

9.1.1.1 Protein Fitness Landscapes

Due to the iterative nature of the optimization cycle that is directed evolution, it is commonly conceptualized as a greedy, uphill walk on a **protein fitness landscape** towards a fitness peak [2]. Each round of mutagenesis and screening searches through the local landscape, typically sampling only a few mutations away from the current position in sequence space. When a hit is identified, a step toward the fitness peak is taken and the local search is repeated, with the entire process continuing until

the fitness is satisfactory or a peak is reached. Importantly, no downward steps into valleys of the fitness landscape are typically allowed in directed evolution.

Fitness landscapes are often visualized as smooth, easy-to-navigate surfaces, but in reality they are discrete, high-dimensional spaces, with many of the dimensions being quite rugged. This ruggedness is due to a phenomenon known in biology as epistasis, where mutational effects are dependent on higher order interactions rather than their individual contributions [15]. Epistasis arises most commonly from direct structural contacts, but interactions between residues can also be modulated by ligands, substrates, allosteric, cofactors, or conformational dynamics [16]. As a result, it is often reasonable to assume that distant mutations are mostly independent, but there are important cases where this assumption breaks down and epistasis must be considered. Otwinowski et al. explored the prevalence of global epistasis, where the mapping of genotype to phenotype showed inherent non-linearity not just due to pairwise interactions between residues [17]. This effect varied in magnitude depending on the protein being studied.

Intertwined with the idea of global epistasis, it is important to consider the prerequisite protein properties that must be satisfied to take fitness measurements, such as expression, stability, and substrate binding. This means that fitness landscapes are a result of some combination of these factors, and changes in any of them can modulate fitness or cause epistasis. For example, Romero and Arnold outline how negative epistasis can arise from a protein stability threshold, where beneficial, but destabilizing mutations combine to completely ablate activity [2]. In fact, any mutation is more likely to be destabilizing than neutral or stabilizing, and therefore, most activity-improving mutations that improve activity are also likely to be destabilizing [17]. Therefore, more stable proteins can be easier to evolve [18]. The protein fitness landscape is a useful analogy for both intrinsic proteins as well as the methods in this chapter, and we come back to this concept often.

9.1.2 Protein Engineering, Directed Evolution, and Machine Learning

In this chapter, we view the protein engineering process through the directed evolution cycle, which adapts natural and artificial selection to the modern laboratory. In this process, the desired properties of engineered proteins are directly queried (assayed) by man-made techniques, instead of interpreted through survival and reproduction (as in natural selection or digital evolution [19]). This allows the engineer to rapidly evolve proteins for desired properties (such as thermal stability or expression) and functions (such as binding or enzymatic activity).

9.1.2.1 The Directed Evolution Cycle

The directed evolution cycle (Fig. 9.1) can be segmented into three basic steps, with two additional steps in cases where protein engineering is driven by machine learning. In Sects. 9.2.1 and 9.2.2, we highlight examples where machine learning has enabled advances in directed evolution, and in Sect. 9.2 we elaborate on the experimental constraints and ML considerations for these steps. The steps are as follows:

1. First, a protein with a measurable amount of the property of interest must be identified. Directed evolution relies on accurate physical interrogation of this scalar value, termed fitness, and a measurable starting point is required to begin this process. Many protein engineering projects fail at this stage, since biochemical ingenuity is required to identify a protein able to accomplish a new goal such as breaking down polyethylene terephthalate (in man-made plastics [20]) or selectively modifying DNA for targeted gene therapy [21] (more on identifying a starting point with ML in Sect. 9.2.1).
2. Second, the protein sequence is randomized to generate a pool of variants, often called a **library**. In nature, this process typically occurs through random mutagenesis, where random mutations in DNA correspond to random changes in the protein sequence, or recombination of existing protein fragments. In the laboratory setting, current experimental techniques give protein engineers more freedom in selecting which sequences to test (more on library generation in Sect. 9.2.2.1).
3. Third, the library is tested for the desired property. This step captures a wide range of assays that have been developed to probe biomolecules. Some examples of approaches capable of generating the largest amounts of data (high-throughput assays), are based on protein fluorescence or binding, and reach hundreds of thousands of labels per month. However, other properties such as enzymatic activity for generating small molecule products are measured in much lower throughput, typically on the scale of hundreds per month. This step often limits the *amount* of data labels the modeler can expect to obtain.

9.1.2.2 How Does Machine Learning Fit into the Directed Evolution Cycle?

In traditional directed evolution, the top k variants (often $k = 1$) are fed back into the second step for further improvement. For methods where directed evolution is further enabled by machine learning, two other steps may follow. The advantages that machine learning confers currently depends on the protein being evolved and its fitness assay. Some protein engineering projects are already able to test millions of variants in a single round, minimizing the value of a machine learning approach. Other projects require a day or more to acquire a single data point and would not even be possible without further guidance from sources such as machine learning.

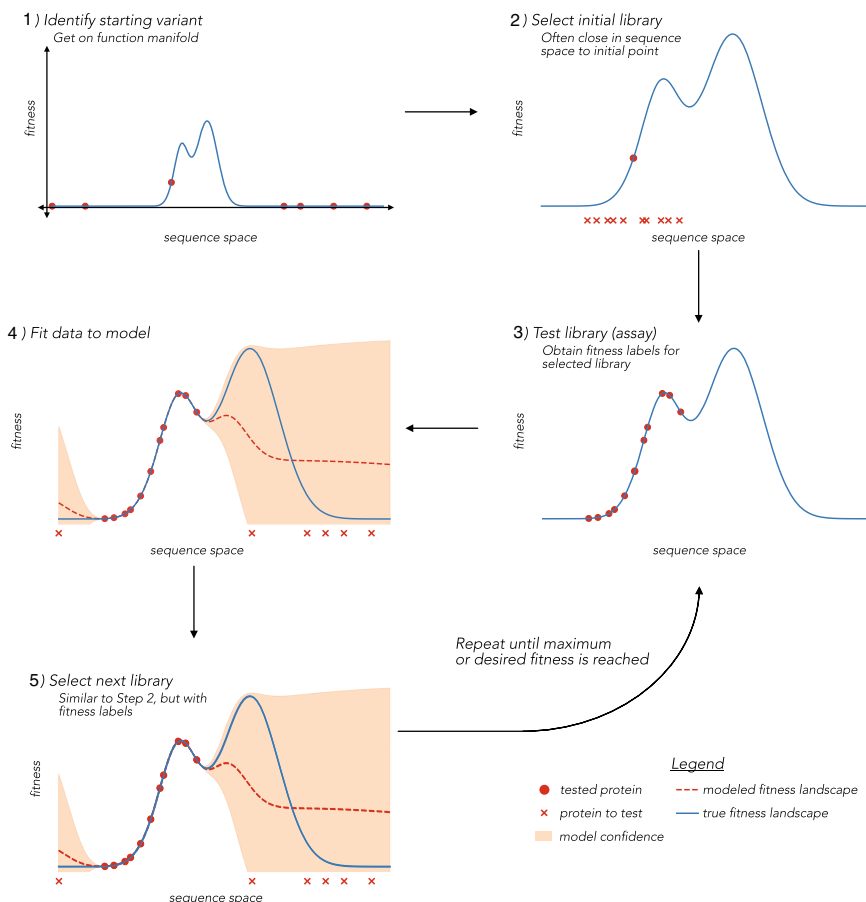


Fig. 9.1 Protein engineering steps viewed as steps in exploring a 1D fitness landscape. In each plot, the protein fitness (scalar measurement of a function or property), is on the y-axis, and the protein sequence is on the x-axis. In (1), a protein with measurable function is identified. In (2), an initial library of protein variants is built. In (3), the library is tested (assayed) for desired property. The next steps are additional for protein engineering methods guided by machine learning, including (4) fitting the models and (5) selecting the next library based on these models. Finally, the process is repeated until a maximum is reached, or the desired fitness satisfies a desired threshold

4. In these machine-learning-assisted approaches, the next step is to fit models to the relationship between proteins and their fitness labels. A wide variety of approaches are available to the machine learning practitioner here, and there are multiple sources of prior knowledge that can be leveraged for proteins. One example is the rich historical record of protein sequences, which can be obtained from sequence databases such as UniRef [22, 23]. From such a database, sets of evolutionarily related sequences (**homologs**) can be obtained and aligned in Multiple Sequence Alignments (**MSAs**), which can be used as priors on viable

sequences. However, while this history represents sequences retained in nature, it does not necessarily represent the distribution of allowed sequences for a specific protein on an engineered task, especially for non-natural activities. For example, mutating the axial ligand of cytochrome P450_{BM3} from a cysteine to a serine unlocked multiple non-natural activities, but an MSA would show high conservation of the cysteine residue and disfavor mutation at this position [24]. However, the exact implications of this for machine learning have not yet been thoroughly examined and more research is needed to elucidate the potential biases of these methods.

5. Finally, these models are used to select optimal proteins for experimental validation. Again, a variety of approaches have been employed in this step such as gradient-based, reinforcement-learning-inspired, and active learning methods (Sect. 9.2.6.3). The sampling strategy often depends on the modeling approach used in the previous step. Additionally, this step is similar to the second in that both may be constrained by current molecular biology techniques. These constraints are often enforced manually, but can be encoded in the design process as well. From here, directed evolution re-enters the third step, and the cycle is repeated until the optimum is reached or design criteria are satisfied.

The process of directed evolution with machine learning is depicted in Fig. 9.1, which interprets each step in the analogy of climbing protein fitness landscapes.

9.2 Applying Machine Learning in Directed Evolution

The directed evolution cycle powered by machine learning methods can be segmented into five steps, as described previously. In this section, we summarize important contributions machine learning has made, and promises to make, to improve over the directed evolution baseline over each of its steps.

Additionally, we discuss experimental constraints to proposed methods, which are imposed by a variety of factors, including available technology, cost, and time. Current laboratory methods for directed evolution have been reviewed extensively [25, 26], so we focus here on facets that can impact the usefulness of machine learning methods. There is often a trade-off between lab-work complexity or cost and efficiency of ML-guided engineering, and we discuss these trade-offs below. Notably, both the input protein sequences and the output experimental labels both depend on physical constraints.

9.2.1 Identifying Starting Variants

The first consideration in a directed evolution experiment is selecting the protein variant to evolve, which can be a nontrivial task. In the most extreme setting, there may be no known proteins that perform the desired function; a more common but related setting is when the desired output of the directed evolution experiment needs to be substantially different from all known proteins due to considerations of scientific novelty or intellectual property.

9.2.1.1 Experimental Considerations for Finding Starting Variants

There are a number of traditional methods for finding starting points for directed evolution, and many of them draw inspiration from Maynard-Smith's observation that functional protein sequences must neighbor other functional sequences in protein space. Thus, when screening for new activities, protein engineers typically begin by searching annotated databases of existing sequences and structures, looking for proteins which perform a similar function to the desired one. Such proteins are hoped to have at least a small level of "promiscuous" activity for the new function. For enzymes, this could mean looking for ones that have the desired mechanism but act on a different substrate. As another approach, plates of variants from previous evolution campaigns might be screened for a new but similar activity. These methods ultimately rely on the latent promiscuity of extant sequences, as the probability that a functional sequence is found through a random search is approximately zero.

9.2.1.2 Applying Machine Learning to Finding Starting Variants

Alternatively, machine learning can be a useful tool for identifying a novel starting variant, typically with weak or suboptimal fitness, that can subsequently be given to a traditional or machine-learning-guided directed evolution pipeline. Existing approaches fall somewhere between using machine learning for finding such variants (1) to screen a large and diverse collection of proteins for any that have a nonzero fitness value [27, 28] or (2) to de-novo design the initial, functional protein [29, 30], with some additional variations summarized in Fig. 9.2.

As an example of the first approach, Shin et al. use an autoregressive language model, trained on approximately 1.2 million natural llama nanobody sequences, to generate a nanobody library that is screened to potentially identify novel binders to a target protein [27]. The generative model enables improved sequence diversity over previous synthetic libraries, enabling the authors to identify new proteins with high binding affinity using an efficient set of approximately 10^5 generated sequences, which is 1000-fold smaller than other libraries.

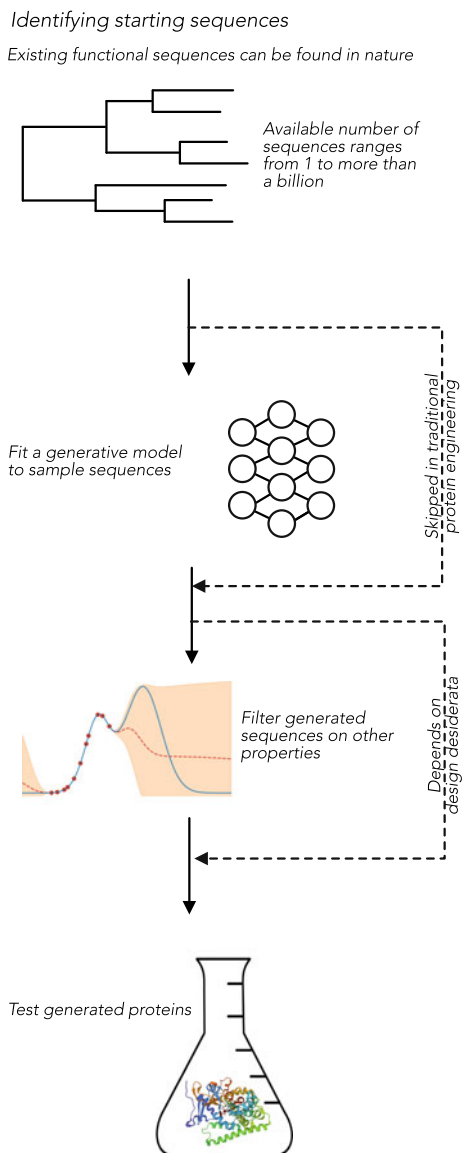


Fig. 9.2 Nature contains a wide variety of natural proteins from which a starting variant can be selected. Machine learning has also been used in various ways to improve the starting variants selected for further protein engineering

The second approach, which promises robust starting points with desired properties without the need to physically screen generated proteins, is an ideal yet unreached. Nonetheless, ProGen [31] is a noteworthy example that has generated functional protein sequences with high (> 50%) success rate on three proteins families [32]. However, there is room for improvement in extending to more difficult functions and protein properties.

These approaches are cases in which the starting variant is unknown or must be substantially different from existing proteins, but this is not a constraint in many other settings where a good, functional variant exists and the goal is simply to improve fitness. Directed evolution could be used directly to evolve the best-known variant, but it may not be the most effective strategy. A known, high-fitness variant may be in a **local** optimum of the fitness landscape such that the evolutionary path to the unknown **global** optimum (or the highest-fitness variant) requires crossing a **fitness valley** (where some intermediate mutations along the path decrease fitness).

Therefore, it may be desirable to begin evolution at a lower-fitness variant from which multiple fitness peaks (both locally- and globally-optimal) are accessible via a greedy uphill walk strategy typical in protein engineering. Engineering a more **evolvable** starting variant, even at some fitness cost, is currently an open question [33] for which machine learning could play an important role. Proteins with high intrinsic stability are thought to be more evolvable [18], and many machine learning models have been developed that either directly (via supervision) or indirectly (as an emergent property of an unsupervised model) predict stability [34–37]. Moreover, we note that directed evolution that begins at a local optimum could still cross fitness valleys via more complex, model-guided strategies beyond greedy exploration, which we review in the following sections.

9.2.2 *Building Initial Libraries*

The data used to train a machine learning model determines what it learns and, by extension, in what situations it can be used to make effective predictions. For protein engineering, this means that the design of the library that will provide training data is critical to the eventual effectiveness of the trained model in finding improved sequences.

9.2.2.1 **Experimental Methods for Library Building**

After a starting point for evolution is identified, a variety of molecular biology methods are available for generating local sequence diversity. While these steps are physically separate from the downstream assay, which couples sequence to a fitness label, the assay throughput is a key factor in selecting the method of diversity generation. We introduce several methods and their biases for the ML practitioner below, but notably, these methods are often combined in protein engineering campaigns.

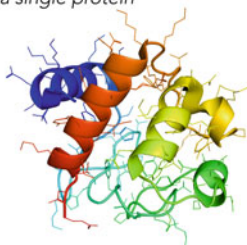
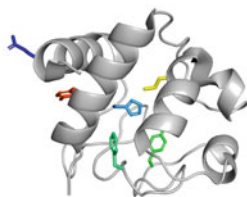
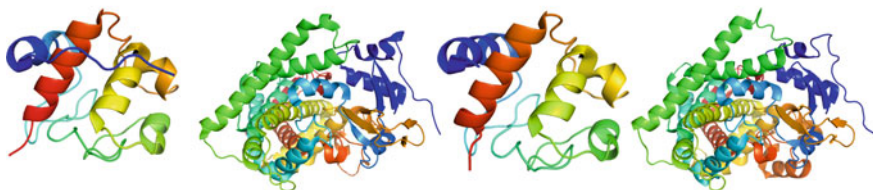
Building initial libraries: Experimental options**A) Random Mutagenesis***Random mutations made from a single protein***B) Site-Saturation Libraries***Explore all possible mutations at specified locations***C) Chimeragenesis***Swap segments of (typically related) parent proteins***D) Direct Gene Synthesis***Full control over generated proteins at the nucleotide level*

Fig. 9.3 Protein sequence diversity can be generated through a combination of various techniques. **a** Random mutagenesis introduced random mutations throughout the protein. **b** Targeted mutagenesis focuses either the available amino acids or the residue positions at which mutations are made. **c** Chimeragenesis combines fragments from various parent proteins. **d** Finally, direct gene synthesis is a relatively new approach that is becoming affordable for protein engineering campaigns

Random mutagenesis (Fig. 9.3a) is one of the most straightforward methods for creating initial sequence diversity. Errors are introduced throughout an initial DNA sequence by either randomly damaging DNA or by introducing errors during replication such as via ePCR, a process that introduces mutations randomly by increasing the error rate of the copying enzyme, the polymerase. However, it is important to note that errors introduced via “random” mutagenesis are not perfectly random in two major ways. First, mutations from one nucleotide to another do not occur at identical frequencies, so the original base can dictate what mutations are most likely at a given position [38]. Second, the genetic code is redundant, with the twenty canonical amino acids encoded by 61, three-nucleotide codons. Although it is possible for multiple nucleotides within a single codon to mutate simultaneously, this is rare, and generally only one nucleotide mutation occurs per codon during random mutagenesis, limiting the mutations available at the amino acid level. In comparison to other mutagenesis methods, the protein engineer has much less control over the generated library—only the rate of mutations can be changed.

Targeted mutagenesis, of which the focused mutagenesis discussed in the previous section is a subset, is an alternative to random mutagenesis that affords more control over the final library. Unlike random mutagenesis, targeted mutagenesis typically assumes that either (1) specific sites in the protein sequence are important to mutate or (2) it is important to be able to access all amino acids at a given position. Site selection usually requires structural knowledge or other biochemical insights into the protein system and, unlike random mutagenesis, any amino acid can be accessed with equal probability. The pool of degenerate oligos can also be modified to achieve a relatively even distribution across all twenty canonical amino acids [39] or to achieve a different distribution of interest [40]. Importantly, as the desired amino acid distribution becomes more complex and more sites are mutated simultaneously, both difficulty of laboratory implementation and the cost of oligos can become untenable, approaching direct gene synthesis costs. Therefore, ML methods relying on targeted mutagenesis for either training set design or evaluation of predicted designs must keep in mind these constraints.

Another strategy for library generation is recombination, which pieces together, or “recombines”, initial diversity into different arrangements to create new diversity. The choice of recombination strategy typically relies on the type of initial diversity on hand. Such diversity could be comprised of a set of functional, **homologous** proteins, the top variants from a random mutagenesis library, or the top variants from a targeted mutagenesis library.

Another recombinatorial approach of note is the use of SCHEMA libraries [41], where fragments of multiple parent proteins (selected by conserving contacts) are swapped, and which has been successfully engineered with machine learning methods [42, 43]. Importantly, some recombination strategies are quite experimentally straightforward and a single round of recombination on top variants can yield much higher improvements in fitness than a single round of random or targeted mutagenesis. Overall, recombination is a very broad category of diversity generation, and due to the array of recombination strategies available, we have only briefly mentioned a few. Further strategies are well reviewed by Packer and Liu [25].

Library design has typically been constrained by methods of library generation, but there is promise to disrupt these traditional paradigms with the advent of cheap gene synthesis technologies [44]. Rather than starting with an initial sequence or pool of sequences and building diversity with mutagenesis or recombination, a set of desired sequences can be synthesized directly for under \$100 per protein—a cost which is currently dropping, particularly for proteins shorter than 100 residues. Ordering pools of sequences with targeted or random mutations is also possible, and emerging synthesis technologies can impart more control over the final distribution. As these DNA synthesis technologies improve, ML methods for protein engineering can begin to leverage and propose precisely-defined libraries that were previously cost-prohibitive.

9.2.2.2 Applying Machine Learning to Library Building

Machine learning models are more effective at making predictions when applied to the same data distribution that was used to train them. In general, for a given **design space** of allowed proteins, this means aiming to fully cover the design space by maximizing the sequence diversity of the training data. Because random collection of sequences from a fixed design space will generally yield diverse sequences, it can be effective for initial training data collection. Random collection of training data is also an attractive approach based on available lab methods (see Sect. 9.2.2.1), and this strategy has been combined with machine learning to engineer halohydrin dehalogenase [45], fluorescent proteins [46, 47], and an adenovirus capsid protein [48].

Unfortunately, experimental data collection is expensive and limited for most applications, making it generally impossible to build a perfect map of a fitness landscape through random collection. This is particularly true for fitness landscapes that cover large swaths of sequence space; a small, randomly selected training dataset is unlikely to contain enough information to build a complete map of the design space. Fortunately, the goal of ML-assisted protein engineering is not to perfectly map the design space, but rather to identify protein variants improved in fitness, as discussed further in Sect. 9.2.5.2. As a result, rather than collecting random training datasets, it can instead be preferable to focus initial data acquisition on regions of the design space believed *a priori* to contain higher-fitness variants. Assuming such focused libraries can be built, a model trained on them will have greater predictive ability in the design regions of interest (those containing high-fitness variants) than others. This approach better aligns with the overall goals of ML-assisted protein engineering, as it allows greater error in prediction of the lowest-fitness variants (where high resolution is not needed—simply knowing that those variants are not fit is good enough) and encourages lower error in prediction of the highest-fitness variants (where there are typically less examples and ranking is more important).

Focused libraries can be particularly helpful when navigating hole-filled protein fitness landscapes [49]. When compared to design spaces built by mutating one position in a protein at a time, **combinatorial** fitness landscapes built by simultaneously mutating multiple positions tend to be enriched in non-functional variants, which

are commonly referred to as “**holes**” in the fitness landscape. These holes are distinct from the fitness **valleys** discussed previously, as holes provide no information about the effect magnitude that mutations have on protein fitness, which is valuable information for training the regression models typically employed for ML-assisted protein engineering.

Wittmann et al. demonstrated that by using so-called zero-shot predictors—models or strategies that can predict protein fitness prior to collection of new experimental data—focused training sets can be constructed that minimize inclusion of holes in training data [49]. Through simulation on a complex, hole-filled, combinatorial fitness landscape, they showed that models trained with these focused training sets tend to be far more effective at identifying the highest-fitness variants than models trained with data drawn randomly from the landscape.

The prior information needed to construct focused libraries can come from many sources. For instance, prediction of protein thermal stability [49], use of meta-predictors of protein fitness [50], or strategies based on evolutionary conservation can all be used to make zero-shot predictions of protein fitness [36, 51–54]. The exact strategy that will be most effective, however, will vary depending on the fitness and protein being optimized. Take, for instance, zero-shot strategies that rely on sequence conservation. Such strategies assume that evolutionary fitness aligns with whatever fitness is being predicted; that is, they assume that mutant proteins more closely resembling known protein sequences (found in databases of protein sequence such as UniProt [55]) are more likely to be functional than others. Should this assumption not hold (for instance, the fitness of a protein being engineered for a new-to-nature activity may not correlate well with evolutionary fitness), or if there are simply not enough homologous protein sequences available to build an effective sequence-based zero-shot prediction model, then the zero-shot predictions are likely to be inaccurate. Inaccurate zero-shot predictions are unhelpful for focused library design; indeed, they may even be detrimental to effective learning by focusing training data collection on regions of the fitness landscape dominated by holes.

Ultimately, the decision between random library design and focused library design will depend on a number of factors. If the fitness landscape to be explored is expected to be minimally complex with few holes and large amounts of training data can be easily collected for it, then random library design is a reasonable approach, as the library itself will be simple to construct, and training data gathered from it will be sufficient to build a comprehensive map of the fitness landscape. If the fitness landscape to be explored is complex, full of holes, or it is challenging to gather training data for it, then focused libraries may be more viable, particularly if high-confidence zero-shot predictions can be made for the fitness landscape. Such libraries may be more challenging to construct in the laboratory, but they will likely result in more efficient machine learning-guided engineering. These are the applications upon which the dropping costs of gene synthesis will have the largest impact.

9.2.3 *Assaying Protein Fitness Creates Labels for Machine Learning*

With proteins performing such a wide variety of different functions, the protein engineering community has had to devise countless different assays to measure them all. As such, assays for protein function vary widely in both accuracy and throughput, with ranges from tens to millions of protein variants (Fig. 9.4). This amount is typically dependent on the project definition of fitness. For instance, if the measurement of fitness can be directly coupled to a sequencing assay (as in deep mutational scanning), then large datasets (10^5 – 10^6) can be rapidly created. Many assays for fitness, however, are limited to comparatively low-throughput chromatographic methods (e.g., HPLC, LCMS, GCMS, etc.), which rely on physical separation of a mixture through a column, producing smaller datasets (10^1 – 10^4). The amount of data available for training a model will dictate how much of sequence space can be explored and how accurate the predictions of fitness for new sequences will be. At the same time, however, the goal of ML in protein engineering is to reduce the burden of experimental screening and expedite the process, so a balance must be found between the amount of data collected and the accuracy of predictions of models trained on that data.

Additionally, one should consider where an ML method for protein engineering may have the biggest impact on efficiency, cost, and time. Protein functions that can be assayed in high-throughput provide more data for training downstream ML models; however, applying an ML-based engineering method to such a function may not impart as much benefit as it would being applied to engineering a function with a low-throughput assay. Therefore, many recent efforts for building ML methods for protein engineering have focused on the **low-N** regime, where few samples are used for training [43, 49, 56–59].

Final considerations, for both the protein engineer choosing an assay and the ML scientist choosing a dataset, are the assay bias and noise. As interest has grown in applying ML to protein engineering, method developers have sought out sequence-fitness datasets with which to benchmark their approaches. This has resulted in the pursuit of large datasets built by multiplexed assays of variant effect like deep mutational scanning (DMS) [60] and using some fitness-related screen or selection such as fluorescence-activated cell sorting (FACS) for a fluorescent protein [61]. Inherent biases in such protein fitness datasets are left under-discussed, making it difficult for the ML community to discern what datasets are appropriate for a given application. As a couple of examples of such biases, DMS fitness measurements can be heavily impacted by the input library and can result in highly non-uniform error across a dataset [62] and FACS-based assays have inherent bias due to fluorescence-based binning. Post-binning sequence processing attempts to smooth the categories into a continuous function, but systemic bias persists, leading Trippe et al. to propose a random gating strategy to reduce it [63]. Noise can be more difficult to address as there tends to be a balance between throughput and measurement accuracy for biological systems. However, examining and quantifying the noise within a dataset

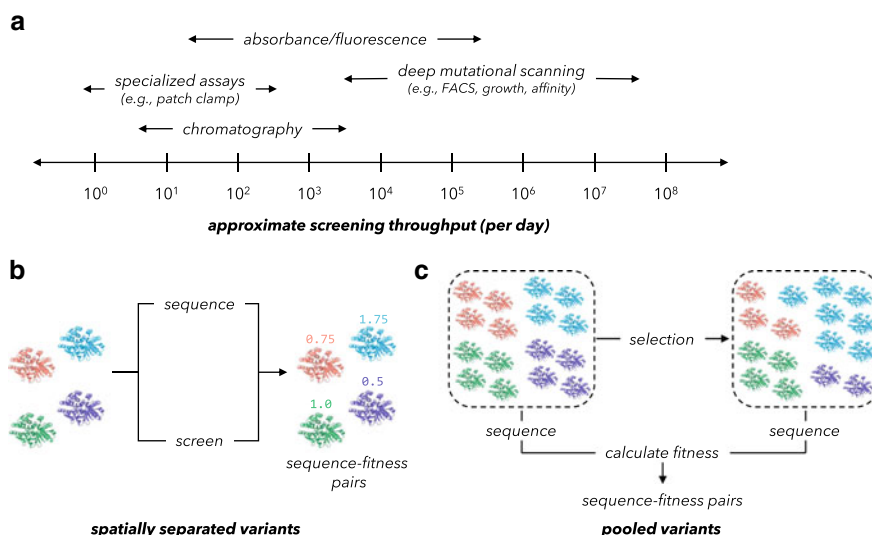


Fig. 9.4 There are many types of screens for protein fitness. **a** Screening throughput can range from less than one variant per day for some specialized experiments to over 10^7 per day for some DMS-based screens. **b** Many protein screens require spatial separation of variants, including many chromatographic methods. This then requires separate sequencing and screening processes that can be later be combined into complete sequence-fitness datasets. **c** With deep-mutational scanning assays sequencing and fitness are linked. Sequencing is performed pre- and post-selection and changes in read frequency are used to calculate a fitness value for every variant

can be important for setting expectations for the success of downstream ML models. This could be done by comparing a subset of high-throughput measurements to more accurate, low-throughput measurements or using published statistical packages for data processing and analysis [62, 64]. Note that all fitness assays have their own noise and bias which should be thoroughly considered when developing an ML method on a particular dataset.

9.2.4 Pairing Sequences to Fitness Data Through Sequencing Creates Features for Machine Learning

Uncommon in the broader ML discipline, datasets that result from typical protein engineering campaigns are label-rich and feature-poor, with many assayed fitnesses and relatively few variant sequences. Because only function is being optimized in directed evolution, this process does not technically require sequencing. Indeed, only the top few variants from each round are sequenced for validation in practice, and sequencing all of the variants is considered an unnecessary, unjustifiable expense. Thus, the remainder of the unimproved variants are discarded without sequencing,

resulting in fitness labels that are rendered useless. Notably, this is not true for libraries where fitness is directly coupled to sequencing, which is a large part of the reason many ML approaches are currently developed on these types of datasets [31, 49, 58, 65, 66].

Variant sequencing, especially for low-throughput assays, has traditionally been done via Sanger sequencing, but this method scales linearly with the number of variants, costing a few dollars per sequence. Depending on the assay used for evaluating protein fitness, sequencing could easily become the most expensive part of a protein engineering campaign. Fortunately, next generation sequencing (NGS) technologies [67] have begun disrupting this paradigm. New sequencing methods that incorporate NGS show promise to continue shifting the sequencing paradigm by spreading reads over many, multiplexed sequences, enabling large-scale sequencing for independently screened fitnesses. For such methods, there is a trade-off between read length and cost per sequence. Both long- and short-read methods can return full-length sequences, but they typically require filling an entire flow cell, which can cost upwards of \$1000 per run and a few dollars per variant [68, 69]. Alternatively, short amplicons from within an entire gene, which are restricted to only a few hundred amino acids, can be sequenced for cents per variant and are a cost-effective compromise when variation is confined to a smaller region of a gene [70]. Nonetheless, both sequencing technologies and sequencing methods for the protein engineering community have been growing rapidly in the past few years, showing promise to address these shortcomings in the near future.

9.2.5 *Training a Protein Sequence-Fitness Model*

Fitness datasets obtained as described above and can be used to fit models of the sequence-fitness relationship. Here, we first describe the methods for representing a protein sequence before diving into the model classes used to predict fitness from sequence.

9.2.5.1 **Representing Protein Sequences**

Proteins are variable-length sequences of twenty canonical amino acids. Due to both the variability in sequence length and the categorical nature of amino acids, an important and long-standing problem has been how to best represent a protein sequence to facilitate machine learning. Roughly speaking, there are two general approaches for doing this: (1) hand-crafting features based on, for example, statistical summaries or biophysical properties of a sequence or its constituent amino acids, and (2) using evolutionarily related sequences to learn likelihoods or embeddings as representations.

Statistical summaries of a sequence: higher-order terms and k-mers. One of the simplest ways to represent a protein sequence is to concatenate one-hot encodings

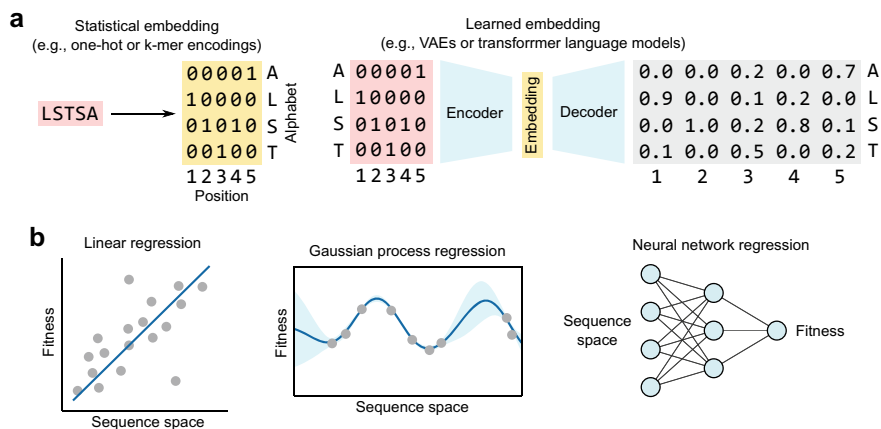


Fig. 9.5 **a** Methods for converting a protein sequence into a representation that can be used as input to a machine learning model include statistical summaries of a sequence (shown here is a cartoon depicting a one-hot encoding, in which sequences are converted to binary values at each sequence position) or learned sequence embeddings, often by training a machine learning model on large collections of sequences (shown here is a cartoon depicting a VAE-based embedding, in which the latent variable learned by a VAE is used as the sequence representation). **b** Once sequence representations have been computed, they can be used as input to supervised models that predict fitness from these representations. These models include linear regression from sequence space to fitness, Gaussian process regression, or a regressor implemented by a neural network

of the amino acid at each position in the sequence (Fig. 9.5a). Despite its simplicity, even a linear model with this representation can be a surprisingly effective baseline in some settings—for example, when we focus on predictive performance on sequences near a wild type [56, 65]. Amino acid similarities may also be directly encoded through matrices based on substitution probabilities of one amino acid for another [71]. Beyond encoding just the amino acid identities at each sequence position, one can also encode the identities of the amino acids at pairs of structurally contacting positions [42, 72].

Generalizing this idea further, one-hot encodings of groups of amino acids, which we call *higher-order terms*—for example, the identities of the amino acids at three positions for a third-order term—have been shown to be both theoretically principled and empirically useful representations for learning protein fitness functions [66, 73–79]. In particular, it has been shown that any fitness function defined over sequences of a fixed length can be expressed exactly as a linear function of all possible higher-order terms of all orders between sequence positions [73]. The number of such terms grows exponentially with sequence length. However, there is increasing evidence that many realistic protein fitness functions can be well-approximated by a linear model of just a small number of these terms [78, 80, 81]; an effective baseline representation may therefore be to one-hot encode all terms up to, for example, second- or third-order [56, 76, 79].

Beyond higher-order terms, the frequencies of different *k*-mers—contiguous subsequences of *k* amino acids—are another kind statistical summary that has been used to represent sequences and learn fitness functions [82, 83].

Physicochemical features. Beyond one-hot encodings of a sequence's residues, or the higher-order terms between them, features based on physicochemical properties of a sequence have been used to learn fitness functions. For example, each amino acid can be featurized as a vector of various physicochemical properties [84, 85] or by a low-dimensional representation thereof [86–88]. These properties can also be combined with structural information [89–91], or with the aforementioned statistical summary representations [82].

However, it not generally clear which physicochemical properties will be relevant for any particular fitness function. Consequently, in recent years considerable attention has turned to *learning* relevant features by fitting low-dimensional representations of evolutionarily related sequences.

Representations learned from evolutionarily related sequences. In directed evolution, one generally starts by identifying a **wild-type** protein (or multiple such wild types) that exists naturally in the genome of some organism and exhibits the function of interest to an appreciable degree. In many cases, that function of interest is needed by more than one organism, and natural selection has yielded multiple homologs residing in a variety of organisms. Though the precise fitness value differs across homologs, due to evolutionary constraints homologs are generally functional as opposed to non-functional. Based on the premise that homologs are mostly functional, a great body of work in recent years has focused on learning the distribution of homologous sequences, as a proxy for the distribution of functional sequences. The density, or approximations thereof, of the resulting learned sequence distribution has been shown to correlate with fitness in many cases, whether modeling the homologs with a hidden Markov model [56, 92, 93], Potts model [53, 94], variational autoencoder [54, 95, 96], or autoregressive model [27], leading to its use as an informative correlate of fitness that does not require any experimental measurements.

Currently, variational autoencoders such as [54] appear to provide the highest correlations with fitness for a variety of fitness datasets, presumably due to their ability to learn evolutionary constraints on higher-order interactions between sequence positions, though autoregressive models [27] and transformers [36] also achieve comparable performance. Due to this correlation, approximations of the density such as the evidence lower bound (ELBO) have shown to be an effective feature when used with supervised learning approaches [56]; the latent space learned by variational autoencoders has also been used to represent proteins for learning fitness functions [95] (Fig. 9.5a). Finally, generative models fit to homologs have also been used to sample novel functional proteins [32, 97, 98].

Beyond homologs, which, by virtue of natural selection, contain sequence patterns necessary for the function of interest, the set of all proteins sequenced so far may similarly reveal patterns necessary for basic properties of all functional proteins, such as the ability to fold into a stable structure. Based on this hypothesis, a growing body of work has trained various deep learning models on massive databases of known proteins, such as UniProt [55], UniRef [23], and UniParc [55], which contain up

to hundreds of millions of sequence as of this writing. The model architectures are almost all borrowed from natural language processing (NLP), ranging from relatively simple skip-gram neural networks [99, 100] and its extensions to embedding text documents [101, 102] to recurrent [57, 103, 104] and convolutional neural networks [105] to, most recently, transformers [31, 36, 106–109]. These models are generally trained using self-supervision tasks, where the goal is, for example, to predict the amino acid at a certain position given the amino acids at other positions (though other approaches such as multi-task learning using function labels [105] and incorporating protein structural information [104] have also been used). The outputs of intermediate layers in these models, which we will call deep embeddings, can then be used to represent protein sequences for learning fitness functions.

Estimates of the density from transformers have also been shown to correlate with a variety of fitness functions [36, 108], presumably both because the model learns sequence patterns required for basic properties of all functional proteins, such as stability, and because the training data contains all homologs corresponding to all known functions. Whether using them for deep embeddings or density estimates, if homologs of a functional wild type are also available for a function of interest, these models can also be further trained or “fine-tuned” on them [32, 57, 108] to focus on a particularly relevant region of sequence space.

Despite promising results on capturing phylogenetic relationships and various clustering tasks, these deep embeddings have shown limited success in learning fitness functions. For example, when used with ridge regression or Gaussian process regression, they do not appear to systematically outperform simpler baselines [56, 65, 102] such as concatenating the aforementioned density of a homolog sequence distribution to a one-hot encoding [56], particularly when there is limited training data.

9.2.5.2 Modeling the Sequence-Fitness Relationship

Given a representation of protein sequences, a variety of model classes have been used to learn the sequence-fitness relationship (Fig. 9.5b). In general, however, note that models that excel under more traditional evaluation metrics—for example, prediction error on held-out sequences from the same distribution as the training data—may or may not be appropriate for use with directed evolution, since in the latter we focus on sequences the model predicts to have high fitness. That is, a model that facilitates directed evolution needs to be reliable even in regions of sequence space far from the training data, particularly in regions on which it predicts high fitness.

With that in mind, a few domain-specific models have been designed with the particular goal of jointly learning (1) a density model over homologs and (2) the fitness function from experimentally labeled sequences [110, 111], under the premise that the density of homologs is positively correlated with the fitness and that therefore both models should stand to gain from sharing information via joint training.

In the more common setting of using only experimentally labeled sequences, two models have been shown to be very strong baselines across a variety of fitness

datasets, particularly in the regime of limited training data (i.e., at most a few hundred labeled sequences): linear regression, for example on one-hot encodings [45, 65, 112], optionally concatenated with the density of a homolog sequence distribution [56], and Gaussian process regression with a variety of kernel functions, which measure the similarity between proteins. These methods include both domain-specific kernels (e.g., structurally informed [42, 72, 113] or mismatch string [102, 114] kernels, which count shared subsequences between proteins) or more general-purpose kernels [43, 46, 83, 91, 102, 115].

Other methods using kernels have been used as well, such as support vector machines for modeling enzyme enantioselectivity [116], membrane protein expression [117], and protein thermostability [90, 118–124]. The latter task, which has been of widespread interest due to its necessity in industrial applications, has also been approached using regression trees and their extensions [118–120].

When labeled sequences are abundant (that is, at least tens to hundreds of thousands), deep learning models can also be effective for learning the fitness function. Models range from simple feedforward networks, which have been used to predict fluorescence [125, 126], histidine synthesis efficacy [127], and viral capsid packaging ability [77], to convolutional neural networks [48, 65, 128, 129], recurrent neural networks [48, 103, 130], and transformers [65, 130], which have been assessed on a variety of protein tasks including fluorescence, thermostability, and binding affinity prediction.

Limitations of current model comparisons. Our understanding of what representations and models are most effective is limited by the constraints of the molecular biology techniques available to generate fitness datasets. For example, the vast majority of currently available fitness datasets only assess the fitnesses of mutants one or two mutations away from a wild type, excluding insertions and deletions [27, 50, 53, 54, 56]. This choice is usually justified by the assumption that most mutations are deleterious [131]; therefore, to generate data with informative, non-zero labels, it is beneficial to stay near sequences already known to be functional. Pragmatically, the resource and throughput constraints on mutagenesis and sequencing protocols described in Sect. 9.2.3 further discourage protein engineers from making more simultaneous mutations. If one intends on generating training data near a functional wild type, comparing representations and models using these datasets may be sufficiently informative. However, beyond this limited setting, fitness datasets with measurements for proteins more broadly dispersed in sequence space are needed to comprehensively understand what machine learning strategies are most effective for learning fitness functions. Dallago et al. [65] have collected several such datasets in recent work; there are also a few datasets with fitness measurements for all possible protein sequences varying at a small number (typically less than twenty) of positions [76, 127, 132], which comprehensively expose the effects of all possible combinations of (albeit a limited number of) mutations.

Current evidence suggests that, when focusing on relatively small regions of sequence space—for example, clustered around a wild type—simple models such as ridge regression on one-hot encodings can be much more effective than more complex models [56, 65], but when considering predictive performance on a broader

region, given sufficient training data there may be benefits from using transformers pretrained on massive protein databases [65]. Further work is needed to systematically investigate the machine learning strategies appropriate for the different types of training and test data that arise in protein engineering.

9.2.6 Protein Selection and Optimization

After obtaining a trained sequence-fitness model, the goal is to use it as part of an **acquisition function** (in the Bayesian optimization sense) to decide which sequences from the theoretical library to characterize next.

9.2.6.1 Specifying the Available Design Space

While calling a sequence-fitness model is much faster than experimental validation, it is still impossible to exhaustively evaluate all possible protein sequences, and the design space must be restricted or implicitly defined. One method of restriction is to limit the **design space** of proteins either by biomolecular or pragmatic constraints. Common examples of design spaces inspired by molecular biology constraints include all single- or double-mutants, all the possible mutations at a small number of sites [49, 58, 132], recombining sequence or structure segments from multiple parent proteins [41, 42, 133, 134], or stochastic libraries that are easy to make using current DNA synthesis techniques [40].

The design space can also be implicitly defined by a *generative model*, allowing examples to be directly generated from the learned distribution. The most straightforward encodings of proteins consider individual or pairs of positions [53, 97]. More recently, researchers have used deep generative models, such as variational autoencoders (VAEs) [54, 135], generative adversarial networks (GANs) [136–138], and autoregressive language models [27, 31, 32, 104, 139], to learn more complex distributions over sequences from which to generate sequences [140].

9.2.6.2 Defining the Acquisition Function

An acquisition function combines sequence information and the sequence-function model to select sequences from the design space for experimental measurements. A simple example of an acquisition function is greedy selection of the top prediction(s) from the sequence-function model. Greedy acquisition is common and can work well in protein engineering [45, 48, 49, 57, 58, 141], but it can become trapped in local optima in more complex fitness landscapes.

Additionally, many standard acquisition functions select only a single example in each round before updating the sequence-fitness function. However, biology techniques are fairly parallelized, and it is often faster and cheaper to obtain data in

batches. Simply acquiring several of the top-ranked sequences according to the acquisition function may result in evaluating many similar sequences, leaving the design space underexplored. Batched acquisition functions are therefore designed to encourage intra-batch diversity [42, 142–146], but this remains an open area of research.

Given a theoretical library and an acquisition function, the simplest method for choosing sequences is to predict the fitness for as many members of the library as is computationally feasible, and then to use those scores to select sequences. However, when the library is defined by a generative model, it can be shifted towards more optimal sequences with *adaptive sampling*, which uses predicted fitness values for previously-generated sequences (from a sequence-fitness model) to re-estimate the parameters of the generative model until convergence [147]. For example, adaptive sampling with a VAE was used to optimize protein expression at the nucleotide level [147] and with a GAN to design antimicrobial peptides [148].

Naive adaptive sampling assumes that the sequence-fitness model is trustworthy; however, these models are inaccurate in many regions of the design space. These regions can be avoided by constraining the generative model to sample sequences close to the training distribution for the sequence-fitness model [126] or by retraining the sequence-fitness model to account for covariate shift [149]. Additionally, sampling from an ensemble of generative models, instead of a single model, also improves generation quality [150, 151].

Similarly, genetic algorithms can be used to produce new sequences [152] by adaptively querying a sequence-fitness model to preserve sequence designs [146]. Alternatively, the sequence-fitness model can be inverted to identify the most desirable of the design space. An inverse of the sequence-fitness model can be trained via an iterative procedure similar to adaptive sampling [153], or, given a *differentiable* sequence-fitness model, can be obtained with gradient-based methods [28, 154, 155].

While it is computationally convenient to exactly specify the sequences to measure next, specifying a new library and sampling from that library may allow many more sequences to be measured for the same cost as directly synthesizing a small number of sequences. This library may be optimized for expected improvement [145], to balance diversity and expected function [77], or to match a desired distribution over sequences [40].

9.2.6.3 Updating the Sequence-Fitness Model with Experimental Knowledge

So far, we have discussed methods for using an acquisition function based on a sequence-fitness model to choose sequences from a theoretical library that are optimized for a desired function. However, in some cases it may be possible to sequentially update the sequence-fitness model with new measurements. The simplest method for sequential optimization is greedy selection of the best-predicted sequence at each step, which is used widely in practice [45]. More sophisticated and

principled methods balance **exploiting** the knowledge in the sequence-fitness model and **exploring** in order to improve the sequence-fitness model [156–159].

Bayesian optimization is a popular framework for sequential optimization [158] that relies on an acquisition function that systematically combines the sequence-fitness model’s prediction with its associated uncertainty to navigate this exploration-exploitation tradeoff. For example, the upper confidence bound (UCB) acquisition function uses a weighted sum of the predicted value and uncertainty, with a larger uncertainty weight encouraging more exploration [157, 160]. UCB is used widely in protein engineering [42, 72, 161, 162] and enjoys strong theoretical guarantees [158]. A popular alternative category of acquisition functions selects the example that is predicted to have the largest improvement in expectation over the current best example or compared to a randomly-drawn example from the previously observed set [163]. Uncertainty can also inform exploration in batch or parallel acquisition, in which multiple protein sequences are acquired and evaluated before updating the sequence-function model [42, 142–145].

Gaussian process models [164] are a popular family of sequence-fitness functions for Bayesian optimization because of their simplicity, flexibility, and strong performance. However, Bayesian neural networks, in which the parameters of the network are themselves random variables with associated prior distributions, can also combine accurate predictions with uncertainty estimates, although efficient and accurate inference in these models can be challenging [165]. One particularly interesting approach is to combine a fitness-space prior (e.g. stability predictions) with the standard neural network weight-space prior [166], and more examples are described in Sect. 9.2.5.2.

Probabilistic sequence-fitness models can also be implemented by model ensembles [167], which train multiple sequence-to-fitness models on the same data and estimate uncertainty from the variance in model predictions [28]. Ensembles are not Bayesian by default, so incorporating prior information into these models can be challenging [168, 169].

Finally, conformal prediction (CP) is an approach for rigorous uncertainty quantification that can be used with any predictive model in a black-box manner, including the aforementioned Gaussian process models or any deep learning model [170–172]. The framework leverages the assumption of exchangeable (e.g., independently and identically distributed) training and test data to construct confidence sets around predictions that provably satisfy *coverage*, a frequentist guarantee on how often the confidence sets contain the true label of test inputs. Beyond exchangeable data, CP has been generalized to handle various forms of distribution shift, including label shift [173], covariate shift [174, 175], distribution shifts in an online setting [176], and test distributions that are close to the training distribution [177]. Notably, in recent work Fannjiang et al. generalized CP for the setting of protein design, in which any of the aforementioned optimization procedures induces a distribution shift wherein the distribution of test (i.e., designed) sequences is dependent on the training data [79]. This method enables practitioners to assess the predictive uncertainty of any

optimization procedure, using any machine learning model, and as such can be a principled approach for selecting an optimization procedure (or any component thereof, including any hyperparameters or the machine learning model class).

9.3 Outlook and Concluding Remarks

In this chapter, we have discussed current efforts to improve protein engineering with machine learning. The field has grown rapidly over the past five years, as the development of deep learning methods and methods to obtain and organize biomolecular data have reached inflection points. We anticipate continued growth in this area of research from both directions and expand on two areas of particular interest here.

9.3.1 *Model Calibration*

The space of possible amino acid combinations is vast, but the space of functional proteins is much smaller, with estimates ranging from one in 10^{11} [178] to as low as one in 10^{77} [179]. The field is regularly identifying new proteins through metagenomic analysis [180], but even if we had a record of all naturally existing proteins, nature may not have completely explored the manifold of functional proteins. Therefore, it is unclear whether characterizing identified natural proteins is sufficient to identify the manifold of functional proteins.

As the goal of protein engineering is to achieve properties beyond what has been thus far observed in proteins, it is particularly critical that machine learning methods navigate sequence space in a way that strikes a balance between exploring new sequences and staying near sequences that a predictive model was trained on, where its predictions tend to be more trustworthy. Methods have characterized and leveraged these trust regions in a variety of ways [57, 79, 126, 150, 181]. In order to characterize more useful trust regions, which will allow for more effective exploration of sequence space, methods for model calibration—accurately quantifying the uncertainty of a model—will become increasingly important. Additionally, there are few combinatorially-comprehensive fitness datasets with which to evaluate existing approaches. A nearly comprehensive GB1 study [132] has been remarkably useful, as have synthetic datasets, but the field would benefit from a wide and diverse set of epistatic fitness landscapes with which to evaluate machine learning methods.

Another reason model calibration is critical is the measurement noise present in fitness data. Common sample preparation methods often introduce noise, may have limited linear range of measurement, and can be difficult to reproduce, particularly for newly developed assays designed to probe more complex biological phenomena. While training methods that account for noisy or subjective labels have gained some traction in other deep learning applications [182, 183], in protein engineering noisy labels are typically only addressed during data pre-processing (e.g., by labeling the

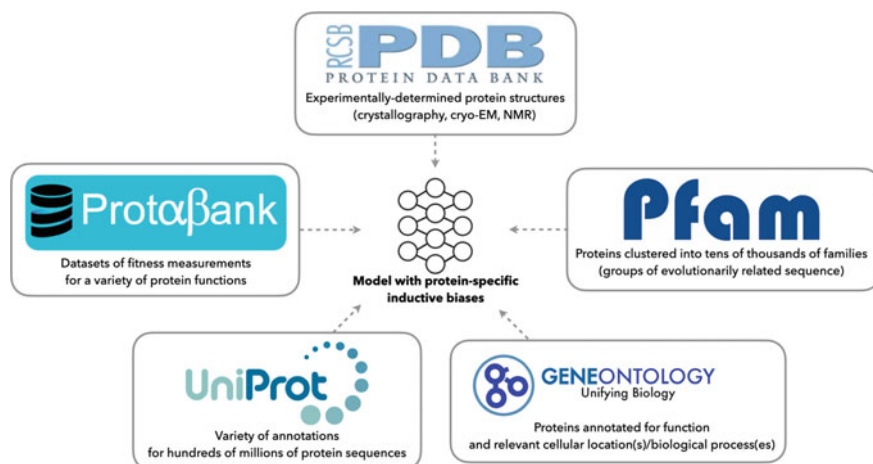


Fig. 9.6 Future machine learning methods for protein engineering will integrate the diverse types of information available from different protein databases, in ways that will improve both predictive performance and calibration in unexplored regions of sequence space

data with Gaussian Mixture Models [27, 48]), although a recent method demonstrates that introducing biophysical priors on the labels can improve predictions [166].

9.3.2 Protein-Specific Data Modalities and Inductive Biases

Much of this chapter is focused on the guided acquisition of data for a particular protein function, which is being collected and curated by ProtαBank [184]. However, the amount of information available for proteins in other data modalities is also rapidly increasing (Fig. 9.6). Many databases exist for various properties of proteins, ranging from protein stability to binding. Of particular note are UniProt [55] and the Protein Data Bank (PDB) [185, 186].

UniProt contains a large amount of information about proteins beyond their sequence, including cross-references to functional labels, disease-association and Protein family (Pfam) classifications at the per-residue level [187], Gene Ontologies on the per-protein example [188, 189], and links to several other databases. UniProt also releases reference protein clusters (UniRef [55, 190]), which are currently almost ubiquitous as the training set for large protein language models [107]. This dataset will continue to grow as more metagenomes are sequenced [180] and more unique proteins are identified. Some approaches to modeling proteins are conditioned on protein functional labels [30] or other data available in UniProt [31], but there is no clear optimal approach to incorporating annotations about all proteins for protein engineering campaigns, which are often focused on specific protein families.

The PDB is the primary source of experimental protein structure data, and it largely consists of static protein structures obtained through protein crystallography, although the number of structures obtained through cryo-EM and NMR are also increasing. Protein structures are invaluable to biologists in providing much-needed context for molecules that are otherwise difficult to probe. However, mutations may have effects that are not captured by static structures. For example, they may bias the protein's Boltzmann distribution toward different conformational states in the ensemble without perturbing the ground state crystal structure, or the crystal structure may simply have too low resolution to capture small changes. Nonetheless, structure can be a useful prior in directed evolution [49, 104] and can also be directly applied to identify beneficial mutations [191].

While protein data continues to grow, methods with inductive biases tailored for biology are comparatively underdeveloped, particular for protein engineering applications. A few notable exceptions have been developed in application to protein structure modeling. For example, the MSA Transformer [108] develops a variant of axial attention specific for MSAs, which contain an aligned homolog sequence in each row. The protein sequences in an MSA likely share similar structure, and the MSA transformer is able to leverage this structure by tying row attention maps.

AlphaFold [13] has the potential to enable protein sequence-based protein engineering methods to incorporate structural information. AlphaFold incorporated several biophysical inductive biases in the CASP14 protein structure prediction contest. These include a variant of attention to account for the triangle inequality on distances, a lowered emphasis on the linear input sequence of a protein (which folds into a three-dimensional structure), and a variant of axial attention for the MSA. These methods have been successful for improving protein structure prediction, and it is likely that machine learning methods adapted from them (or even the predictions themselves) will improve each step of the protein engineering cycle.

9.3.3 Summary

Protein engineering is an optimization strategy, and many recent advances in machine learning have been successfully applied or developed to the directed evolution of proteins. In this chapter, we have highlighted studies where machine learning contributes to each step of the optimization process. As we continue to gather more protein information and develop methods to bootstrap and guide engineering from these data, machine learning will play an increasingly important role in protein engineering.

Acknowledgements This work was supported by an NSF Division of Chemical, Bioengineering, Environmental, and Transport Systems (CBET 1937902). Individual support for K.E.J. was provided by Amazon AI4Science.

References

1. Arnold FH (1998) Design by directed evolution. *Acc Chem Res* 31:125–131
2. Romero PA, Arnold FH (2009) Exploring protein fitness landscapes by directed evolution. *Nat Rev Mol Cell Biol* 10:866–876. <https://doi.org/10.1038/nrm2805>
3. Ching T et al (2018) Opportunities and obstacles for deep learning in biology and medicine. *J Roy Soc Interface* 15:20170387
4. Coley CW, Eyke NS, Jensen KF (2019) Autonomous discovery in the chemical sciences part II: outlook. *Angew Chem Int Ed* 59(52):23414–23436
5. Zou J et al (2019) A primer on deep learning in genomics. *Nat Genet* 51:12–18
6. Eraslan G, Avsec Ž, Gagneur J, Theis FJ (2019) Deep learning: new computational modelling techniques for genomics. *Nat Rev Genet* 20:389–403
7. Vamathevan J et al (2019) Applications of machine learning in drug discovery and development. *Nat Rev Drug Discov* 18:463–477
8. Gao W, Mahajan SP, Sulam J, Gray JJ (2020) Deep learning in protein structural modeling and design. *Patterns* 1:100142
9. Noé F, De Fabritiis G, Clementi C (2020) Machine learning for protein folding and dynamics. *Curr Opin Struct Biol* 60:77–84
10. AlQuraishi M (2021) Machine learning in protein structure prediction. *Curr Opin Chem Biol* 65:1–8
11. Greslehner GP (2018) What do molecular biologists mean when they say ‘structure determines function’?
12. Ovchinnikov S, Huang P-S (2021) Structure-based protein design with deep learning. *Curr Opin Chem Biol* 65:136–144
13. Jumper J et al (2021) Highly accurate protein structure prediction with AlphaFold. *Nature* 596:583–589
14. Maynard Smith J (1970) Natural selection and the concept of a protein space. *Nature*. <https://doi.org/10.1038/225563a0>
15. Starr TN, Thornton JW (2016) Epistasis in protein evolution. *Protein Sci* 25:1204–1218
16. Miton CM, Buda K, Tokuriki N (2021) Epistasis and intramolecular networks in protein evolution. *Curr Opin Struct Biol Eng Des Membr* 69:160–168. <https://doi.org/10.1016/j.sbi.2021.04.007>
17. Otwinowski J, McCandlish DM, Plotkin JB (2018) Inferring the shape of global epistasis. *Proc Nat Acad Sci* 115:E7550–E7558. <https://doi.org/10.1073/pnas.1804015115>
18. Bloom JD, Labthavikul ST, Otey CR, Arnold FH (2006) Protein stability promotes evolvability. *Proc Nat Acad Sci U S A* 103:5869–5874
19. Lehman J et al (2020) The surprising creativity of digital evolution: a collection of anecdotes from the evolutionary computation and artificial life research communities. *Artif Life* 26:274–306
20. Austin HP et al (2018) Characterization and engineering of a plastic-degrading aromatic polyesterase. *Proc Nat Acad Sci* 115:E4350–E4357
21. Waehler R, Russell SJ, Curiel DT (2007) Engineering targeted viral vectors for gene therapy. *Nat Rev Genet* 8:573–587
22. Suzek BE, Huang H, McGarvey P, Mazumder R, Wu CH (2007) UniRef: comprehensive and non-redundant UniProt reference clusters. *Bioinformatics* 23:1282–1288
23. Suzek BE et al (2015) UniRef clusters: a comprehensive and scalable alternative for improving sequence similarity searches. *Bioinformatics* 31:926–932
24. Hyster TK, Arnold FH (2015) P450BM3-axial mutations: a gateway to non-natural reactivity. *Israel J Chem* 55:14–20. <https://doi.org/10.1002/ijch.201400080>
25. Packer MS, Liu DR (2015) Methods for the directed evolution of proteins. *Nat Rev Genet* 16:379–394
26. Currin A, Swainston N, Day PJ, Kell DB (2015) Synthetic biology for the directed evolution of protein biocatalysts: navigating sequence space intelligently. *Chem Soc Rev* 44:1172–1239